Os dois arquivos definem a lógica central do **GildedRose Kata**. O comportamento do sistema está quase todo no método `updateQuality()` da classe `GildedRose`. A classe `Item` apenas representa os dados.

A geração de uma **suíte de testes unitários** deve seguir um processo estruturado:

1. Analisar regras de negócio implícitas no código.
2. Identificar classes de comportamento diferentes.
3. Criar cenários que exercitem cada regra.
4. Transformar os cenários em testes automatizados.
5. Garantir cobertura das condições principais.

A seguir está o passo-a-passo.

---

1. Análise da estrutura do código

Classe `Item`

A classe possui apenas três atributos:

- `name`
- `sellIn`
- `quality`

Ela não contém lógica, apenas estado. Portanto **não precisa de testes complexos**, apenas é usada como entrada/saída nos testes.

Classe `GildedRose`

O método relevante é:

```
public void updateQuality()
```

Ele:

- percorre todos os itens
- atualiza `sellIn`
- atualiza `quality`
- possui **regras diferentes dependendo do nome do item**

Isso indica que **cada tipo de item deve gerar cenários de teste distintos**

Isso indica que **cada tipo de item deve gerar cenários de teste distintos**.

---

2. Identificação das regras de negócio

Lendo o código é possível extrair as regras principais.

Regra 1 — Item normal

Itens que **não são especiais**:

- Quality diminui 1 por dia
- Depois do vencimento (`sellIn < 0`) diminui 2
- Quality nunca fica abaixo de 0

Itens normais são todos exceto:

- Aged Brie
- Backstage passes
- Sulfuras

Regra 2 — Aged Brie

- Quality **aumenta** com o tempo
- Máximo = 50
- Após vencimento aumenta **2**

Regra 3 — Backstage passes

Quality aumenta conforme o show se aproxima:

SellIn	Aumento
>10	+1
<=10	+2
<=5	+3

Depois do show:

```
quality = 0
```

```
quality = 0
```

Regra 4 — Sulfuras

Item lendário:

- quality nunca muda
- sellIn nunca muda

Regra 5 — Limites de qualidade

Sempre:

```
0 <= quality <= 50
```

Exceto Sulfuras (normalmente 80).

---

3. Identificação dos cenários de teste

Agora transformamos regras em **casos de teste**.

Itens normais

Casos necessários:

1. diminui quality em 1
2. diminui sellIn
3. quality nunca negativa
4. após vencimento diminui 2

Aged Brie

Casos:

1. aumenta quality
2. aumenta 2 após vencimento
3. não passa de 50

não passa de 50

Backstage passes

Casos:

1. +1 quando sellIn > 10
2. +2 quando sellIn <=10
3. +3 quando sellIn <=5
4. quality vira 0 após show

Sulfuras

Casos:

1. sellIn não muda
2. quality não muda

---

4. Estrutura da suíte de testes

Usaremos **JUnit 5**.

Estrutura:

```
src/test/java/com/gildedrose/GildedRoseTest.java
```

---

5. Implementação dos testes

```
package com.gildedrose;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class GildedRoseTest {

    private Item update(Item item) {
        Item[] items = new Item[] { item };
        GildedRose app = new GildedRose(items);
        app.updateQuality();
```

```
        app.updateQuality();
        return items[0];
    }

    // ---------------------------
    // Itens normais
    // ---------------------------

    @Test
    void normalItem_decreasesQuality() {
        Item item = update(new Item("Normal Item", 10, 20));

        assertEquals(19, item.quality);
    }

    @Test
    void normalItem_decreasesSellIn() {
        Item item = update(new Item("Normal Item", 10, 20));

        assertEquals(9, item.sellIn);
    }

    @Test
    void normalItem_qualityNeverNegative() {
        Item item = update(new Item("Normal Item", 10, 0));

        assertEquals(0, item.quality);
    }

    @Test
    void normalItem_degradesTwiceAfterExpiration() {
        Item item = update(new Item("Normal Item", 0, 10));

        assertEquals(8, item.quality);
    }

    // ---------------------------
    // Aged Brie
    // ---------------------------

    @Test
    void agedBrie_increasesQuality() {
        Item item = update(new Item("Aged Brie", 10, 10));

        assertEquals(11, item.quality);
    }

    @Test
    void agedBrie_increasesTwiceAfterExpiration() {
        Item item = update(new Item("Aged Brie", 0, 10));

        assertEquals(12, item.quality);
    }
```

```
    @Test
    void agedBrie_neverExceeds50() {
        Item item = update(new Item("Aged Brie", 10, 50));

        assertEquals(50, item.quality);
    }

    // ---------------------------
    // Backstage passes
    // ---------------------------

    @Test
    void backstagePass_increaseBy1_whenMoreThan10Days() {
        Item item = update(new Item(
            "Backstage passes to a TAFKAL80ETC concert", 15, 10));

        assertEquals(11, item.quality);
    }

    @Test
    void backstagePass_increaseBy2_when10DaysOrLess() {
        Item item = update(new Item(
            "Backstage passes to a TAFKAL80ETC concert", 10, 10));

        assertEquals(12, item.quality);
    }

    @Test
    void backstagePass_increaseBy3_when5DaysOrLess() {
        Item item = update(new Item(
            "Backstage passes to a TAFKAL80ETC concert", 5, 10));

        assertEquals(13, item.quality);
    }

    @Test
    void backstagePass_qualityDropsToZeroAfterConcert() {
        Item item = update(new Item(
            "Backstage passes to a TAFKAL80ETC concert", 0, 10));

        assertEquals(0, item.quality);
    }

    // ---------------------------
    // Sulfuras
    // ---------------------------

    @Test
    void sulfuras_neverChangesQuality() {
        Item item = update(new Item(
            "Sulfuras, Hand of Ragnaros", 10, 80));

        assertEquals(80, item.quality);
```

```
    }

    @Test
    void sulfuras_neverChangesSellIn() {
        Item item = update(new Item(
            "Sulfuras, Hand of Ragnaros", 10, 80));

        assertEquals(10, item.sellIn);
    }
}
```

---

6. Por que esses testes são suficientes

Eles cobrem todas as **decisões principais do método**:

Regra	Teste
Item normal	✔
Degradação após vencimento	✔
Quality mínima	✔
Aged Brie aumenta	✔
Aged Brie limite 50	✔
Backstage regras 10 dias	✔
Backstage regras 5 dias	✔
Backstage pós-show	✔
Sulfuras imutável	✔

Esses testes exercitam praticamente **todos os ramos condicionais do método**, que é exatamente o objetivo antes de iniciar o **refactoring** no kata.

---

7. Estratégia usada para escolher os testes

A seleção seguiu três princípios de engenharia de testes:

1. Particionamento de equivalência

Cada tipo de item é uma classe diferente:

- Normal
- Brie
- Backstage
- Sulfuras

---

2. Análise de limites

Valores críticos:

```
sellIn = 10
sellIn = 5
sellIn = 0
quality = 0
quality = 50
```

---

3. Cobertura de ramos

Cada `if` do método gera um cenário.

Se quiser, posso também mostrar:

- **a matriz de decisão completa do método `updateQuality()`**
- **como alcançar ~100% de cobertura com JaCoCo**
- **como refatorar o código após garantir os testes (Strategy Pattern)** — que é a etapa clássica do GildedRose Kata.